

CNNs

Clase práctica 16

Repaso de CNN | ¿Por qué?

Los **píxeles** en una imagen **no son independientes** entre sí. Cada pixel está relacionado con sus vecinos.

¿Una red densa puede representar esta relación espacial?

Queremos captar patrones y características a **distintos niveles** para luego aprender cómo se relacionan entre sí.

Repaso de CNN | Conceptos

Un **kernel** es una matriz que se utiliza para detectar características como bordes, texturas o patrones. Se desliza sobre la imagen, aplicando operaciones de convolución (en unos min. lo vemos).

El **pooling** permite reducir la dimensionalidad y “resumir” características de la imagen, capturando lo esencial de la misma. Combinando capas de convolución y pooling, se captan características más complejas.

Los **canales** representan las distintas características filtradas de una imagen. Permiten a la red procesar múltiples aspectos a la vez.

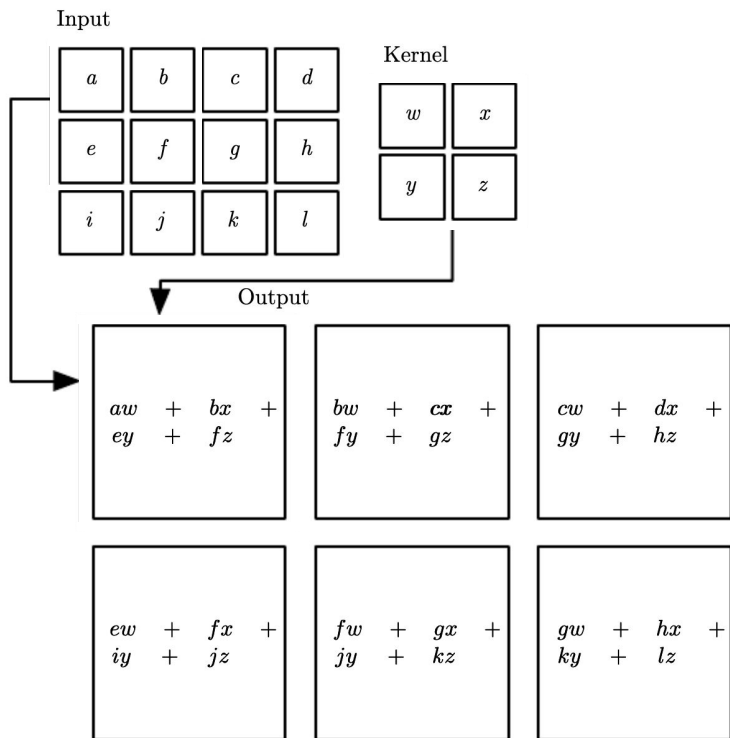
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

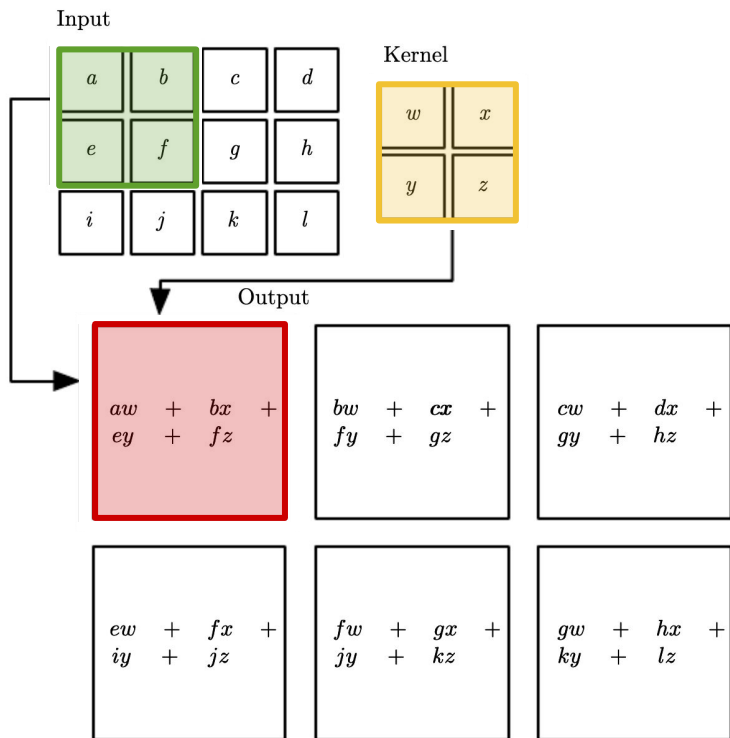
4		

Convolved
Feature

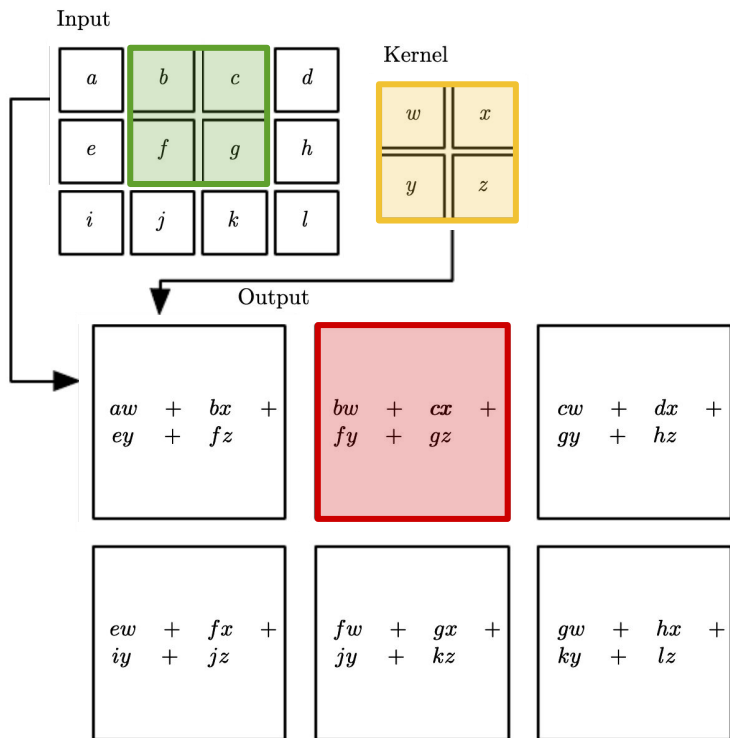
Repaso de CNN | Una convolución



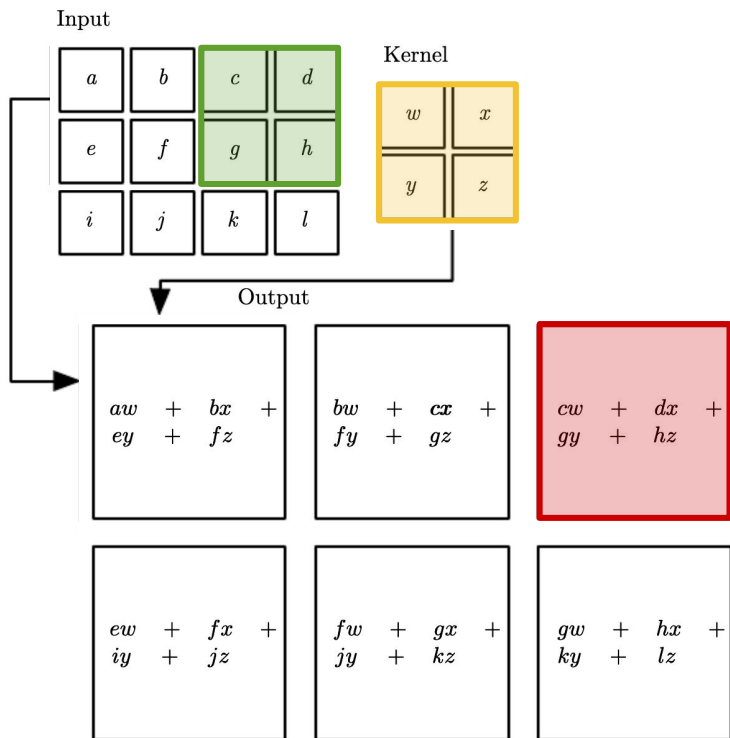
Repaso de CNN | Una convolución (continuación)



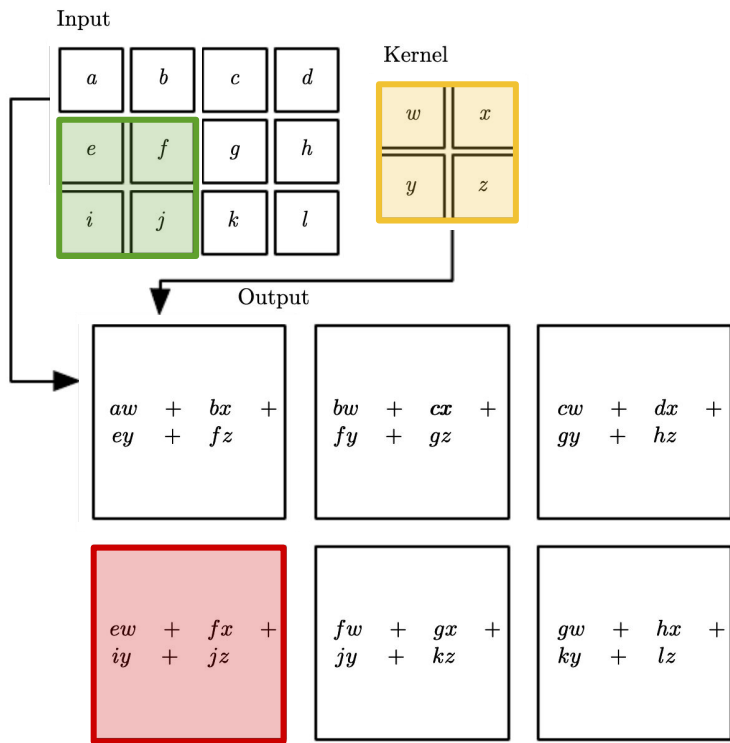
Repaso de CNN | Una convolución (continuación)



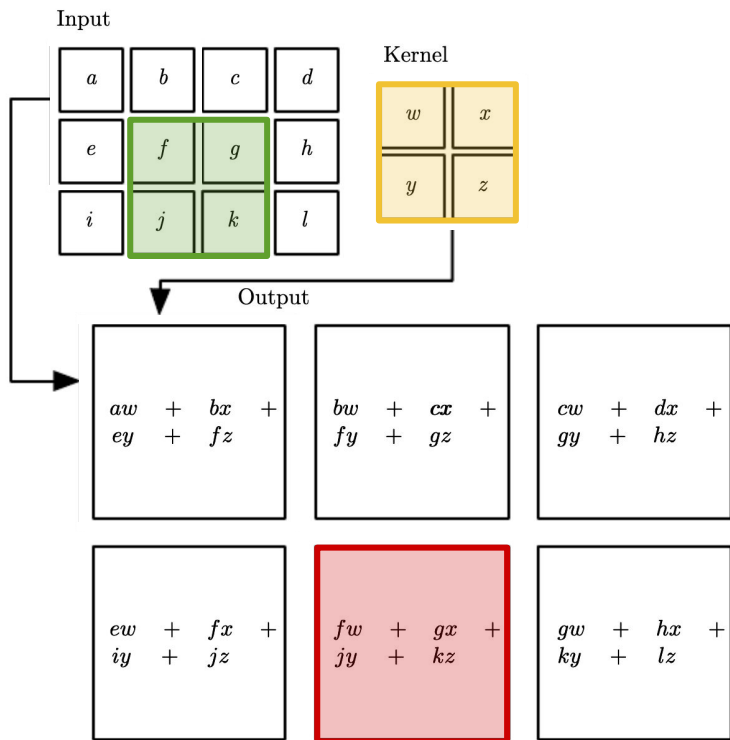
Repaso de CNN | Una convolución (continuación)



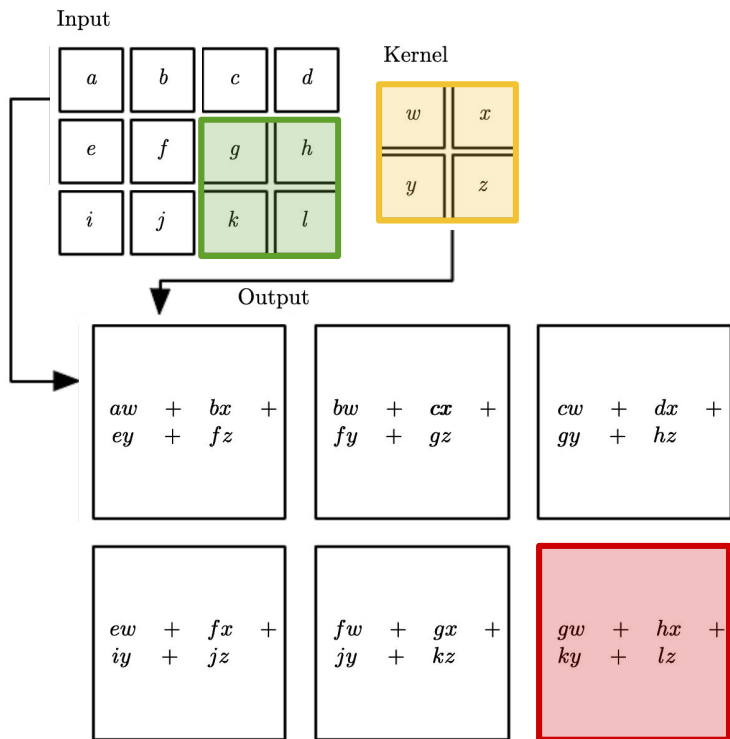
Repaso de CNN | Una convolución (continuación)



Repaso de CNN | Una convolución (continuación)



Repaso de CNN | Una convolución (continuación)

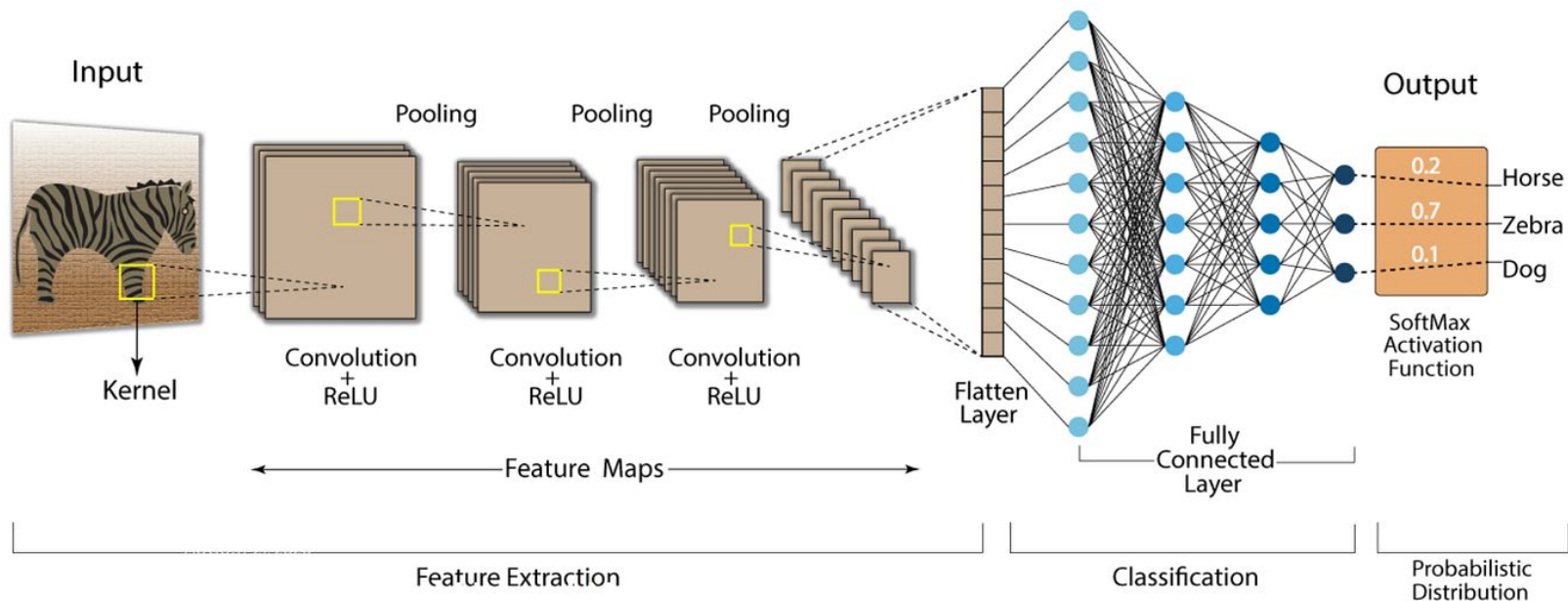


Repaso de CNN | Ejemplo visual interactivo

Vayamos a <https://setosa.io/ev/image-kernels/>.

Repaso de CNN | Panorama

Convolution Neural Network (CNN)



Implementación en PyTorch

Vamos a usar `torch.nn.Conv2D`. Veamos su [documentación oficial](#).

in_channels: el número de canales de entrada. Por ej., en una imagen RGB, este valor es 3 (rojo, verde y azul).

out_channels: el número de kernels a usar. Determina cuántos canales de características queremos extraer.

kernel_size: el tamaño de la matriz del kernel. Por ej., 3 para kernel de 3x3.

stride: número de píxeles a mover el kernel en cada paso.

padding: cantidad de píxeles a agregar en los márgenes antes de aplicar la convolución.

padding_mode: qué técnica emplear para asignar valores a los píxeles agregados.

Repaso de CNN | Panorama

```
model = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=9, padding=●),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
  
    nn.Conv2d(32, 64, kernel_size=3, padding=●),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
  
    nn.Flatten(),  
  
    nn.Linear(●, 128),  
    nn.ReLU(),  
  
    nn.Linear(128, 26)  
)
```

Implementación en PyTorch (continuación)

En línea, hay varios **recursos** vinculados a dichos parámetros que son muy útiles.

Algunos de ellos son los siguientes:

- <https://poloclub.github.io/cnn-explainer/>
- <https://madebyollin.github.io/convnet-calculator/>
- <https://cs231n.github.io/convolutional-networks/>
- <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>